Week 2 - Preprocessing, part 2

1. Lesson: None

2. Weekly graph question

The Storytelling With Data book mentions planning on a "Who, What, and How" for your data story. Write down a possible Who, What, and How for your data, using the ideas in the book.

By utilizing data science, we can employ models to detect financial crime and fraud detection. Financial services companies such as credit card and loan issuers can utilize these models to ensure customers are not charged for items they did not purchase, and loans are not requested for invalid use cases.

Who: Business stakeholders

What: Employing a data science and engineering team and deploying detection models will identify and prevent X amount of fraudulent charges and loan applications

How: Historical credit card transaction data, loan application data, and customer data will be utilized to train models. These models will be deployed to detect future financial crime and fraud on future transactions and applications with an alerting system and manual or automated intervention workflows.

3. Homework - work with your own data

In [1]:
```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
```

This week, you will do the same types of exercises as last week, but you should use your chosen datasets that someone in your class found last semester. (They likely will not be the particular datasets that you found yourself.)

Here are some types of analysis you can do Use Google, documentation, and ChatGPT to help you:

- Summarize the datasets using info() and describe()

- Are there any duplicate rows?
- Are there any duplicate values in a given column (when this would be inappropriate?)
- What are the mean, median, and mode of each column?
- Are there any missing or null values?
  - Do you want to fill in the missing value with a mean value? A value of your choice? Remove that row?
- Identify any other inconsistent data (e.g. someone seems to be taking an action before they are born.)
- Encode any categorical variables (e.g. with one-hot encoding.)

Conclusions:

- Are the data usable? If not, find some new data!
- Do you need to modify or correct the data in some way?
- Is there any class imbalance? (Categories that have many more items than other categories).

Loading the Data

In [2]:
```
# https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud
df = pd.read_csv(f"C:/Users/caleb/Downloads/archive/creditcard.csv")
```

In [3]:
```
df
```

Out[3]:

	Time	V1	V2	V3	V4	V5	V6	
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.2395
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.0788
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.7914
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.2376
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.5929
...	...	...	...	...	...	...	...	
284802	172786.0	-11.881118	10.071785	-9.834783	-2.066656	-5.364473	-2.606837	-4.9182
284803	172787.0	-0.732789	-0.055080	2.035030	-0.738589	0.868229	1.058415	0.0243
284804	172788.0	1.919565	-0.301254	-3.249640	-0.557828	2.630515	3.031260	-0.2968
284805	172788.0	-0.240440	0.530483	0.702510	0.689799	-0.377961	0.623708	-0.6861
284806	172792.0	-0.533413	-0.189733	0.703337	-0.506271	-0.012546	-0.649617	1.5770

284807 rows × 31 columns

Summarizing the Data

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

There are no missing values. The data types also look correct for every feature except "Time", which should be treated as an integer or datetime instead.

In [5]:
```
df.describe()
```

Out[5]:

	Time	V1	V2	V3	V4	
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.8480
mean	94813.859575	1.168375e-15	3.416908e-16	-1.379537e-15	2.074095e-15	9.6040
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00	1.3802
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00	-1.1374
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01	-6.9159
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02	-5.4335
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01	6.1192
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01	3.4801

8 rows × 31 columns

As discussed in the documentation, the features V1-V28 are the result of PCA transformation of the original data to protect client confidentiality. Time, Amount, and Class are the other features, which all have different scales. Class is the intended target variable.

Identifying Duplicates

In [6]:

```
# Identify any identical rows
duplicates = df[df.duplicated(keep=False)]
duplicates
```

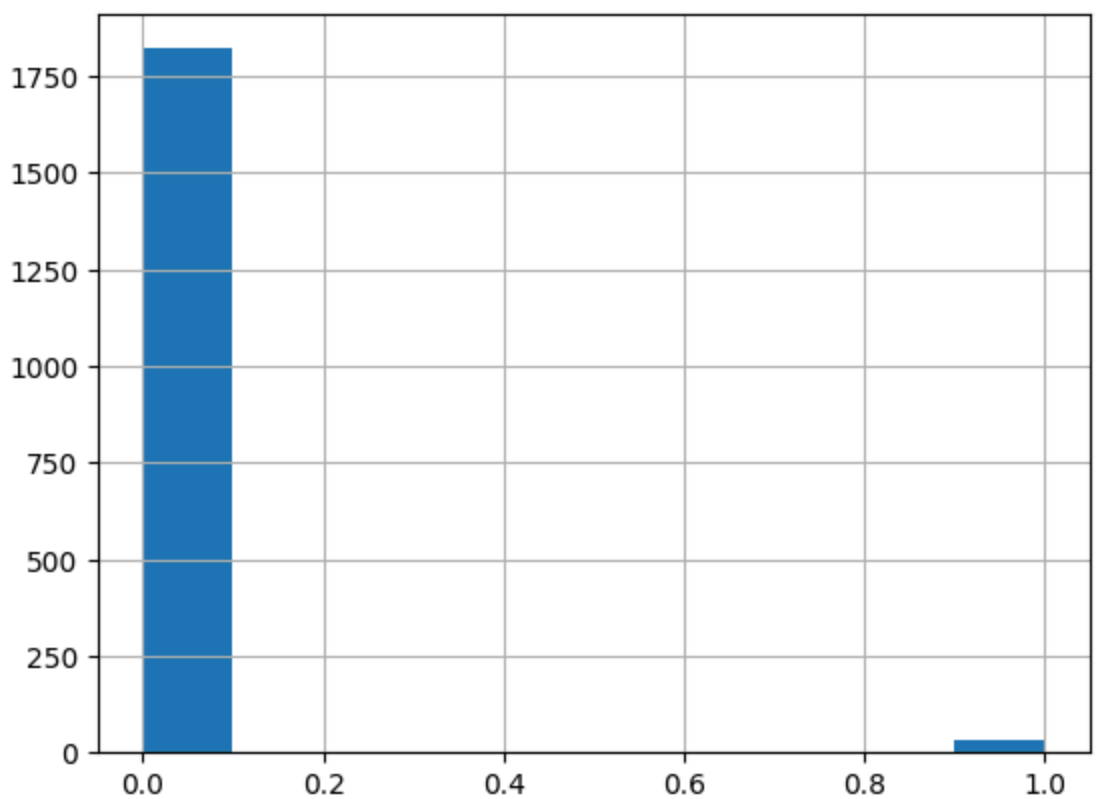
Out[6]:

	Time	V1	V2	V3	V4	V5	V6	V7
32	26.0	-0.529912	0.873892	1.347247	0.145457	0.414209	0.100223	0.711206
33	26.0	-0.529912	0.873892	1.347247	0.145457	0.414209	0.100223	0.711206
34	26.0	-0.535388	0.865268	1.351076	0.147575	0.433680	0.086983	0.693039
35	26.0	-0.535388	0.865268	1.351076	0.147575	0.433680	0.086983	0.693039
112	74.0	1.038370	0.127486	0.184456	1.109950	0.441699	0.945283	-0.036715
...	...	...	...	...	...	...	...	...
283485	171627.0	-1.457978	1.378203	0.811515	-0.603760	-0.711883	-0.471672	-0.282535
284190	172233.0	-2.667936	3.160505	-3.355984	1.007845	-0.377397	-0.109730	-0.667233
284191	172233.0	-2.667936	3.160505	-3.355984	1.007845	-0.377397	-0.109730	-0.667233
284192	172233.0	-2.691642	3.123168	-3.339407	1.017018	-0.293095	-0.167054	-0.745886
284193	172233.0	-2.691642	3.123168	-3.339407	1.017018	-0.293095	-0.167054	-0.745886

1854 rows × 31 columns

In [7]:
```
duplicates['Class'].hist()
```

Out[7]: <Axes: >



In [8]:
```
len(duplicates[duplicates['Class'] == 1])
```

Out[8]: 32

In [9]:
```
len(df[df['Class'] == 1])
```

Out[9]: 492

In [10]:
```
32 / 492
```

Out[10]: 0.06504065040650407

If we remove the duplicates it would result in losing 6% of the positive class (fraudulent charges). I will take the assumption that it is important to keep all these transactions instead of removing them. It is worth further investigation as to why there could be duplicate rows (especially with identical timestamps)

Identifying Inconsistent Data

Amounts

In [11]:
```
df['Amount'].min(), df['Amount'].max()
```

Out[11]: (np.float64(0.0), np.float64(25691.16))

In [12]:
```
len(df[df['Amount'] == 0])
```

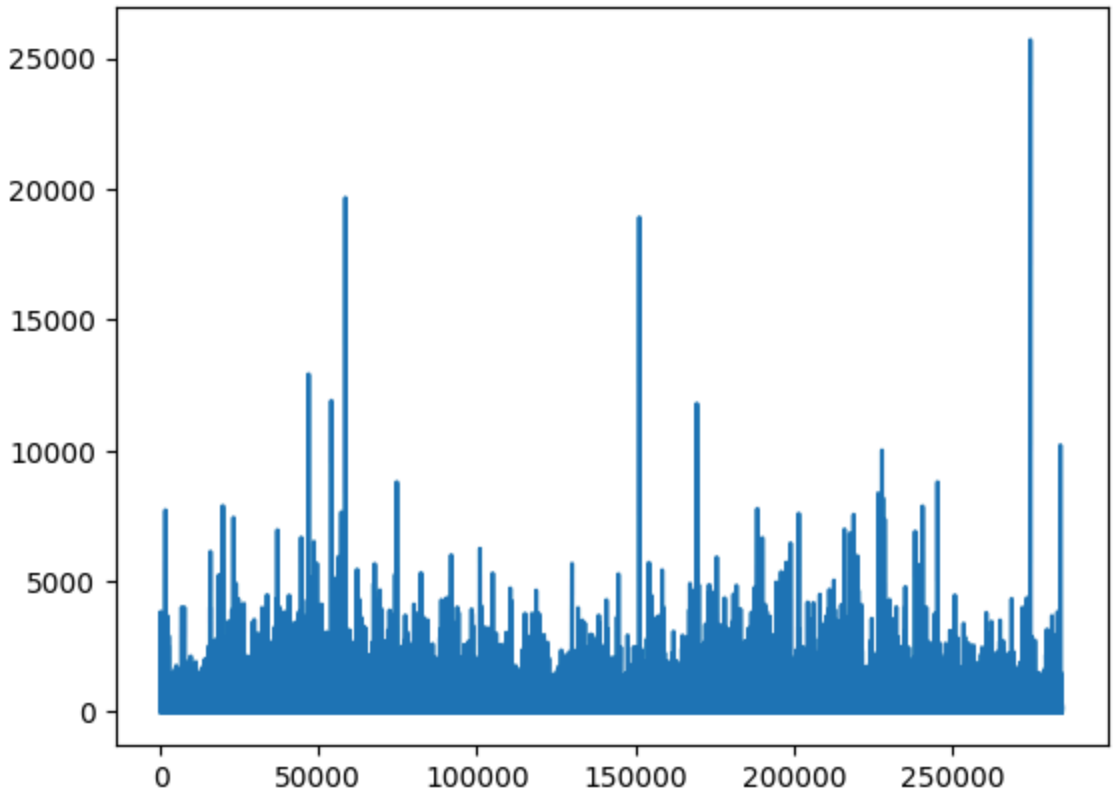
Out[12]: 1825

In [13]:
```
len(df[df['Amount'] == 25691.16])
```

Out[13]: 1

In [14]:
```
# Check whether any invalid amounts are present
df['Amount'].plot()
```

Out[14]: <Axes: >



The smallest amount is 0 and the max is 25,691.16. The largest Amount seems huge for a single credit card transaction, but since there are many it seems valid. The minimum of 0 seems unlikely as well, but it could be a charge to check a card is active. I am assuming they are valid as well.

Labels

In [15]:
```
df['Class'].unique()
```

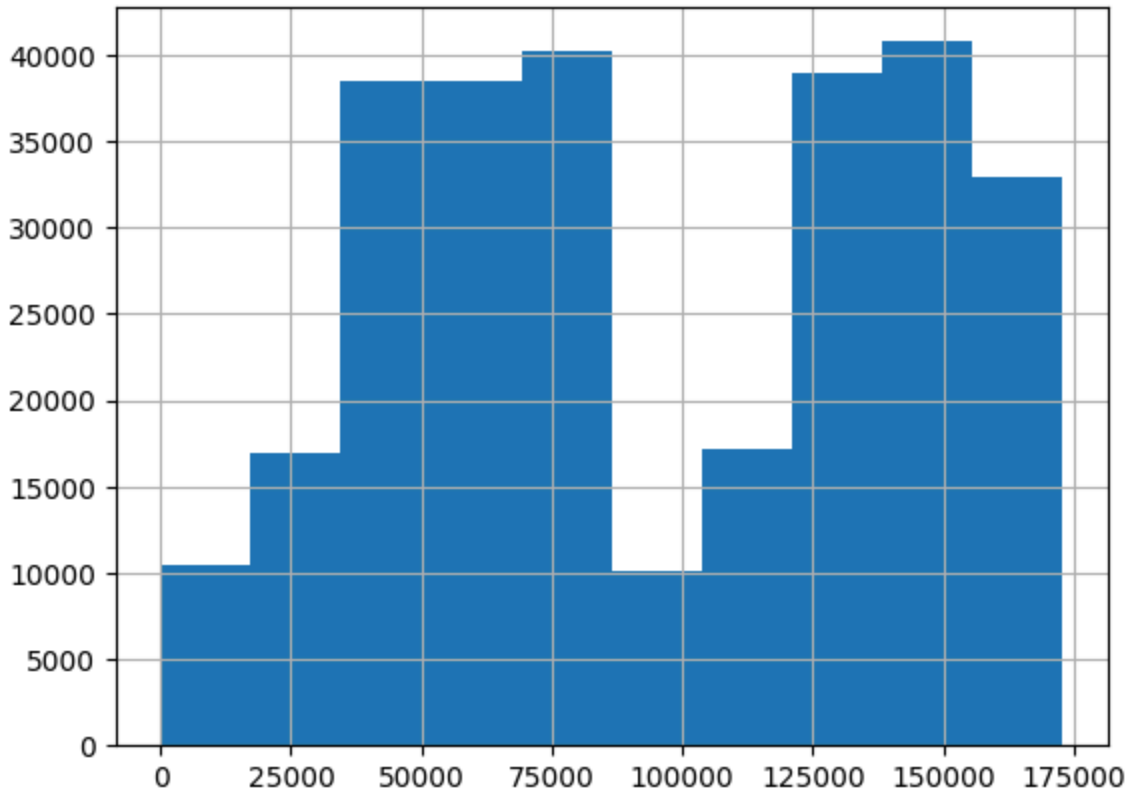
Out[15]:
```
array([0, 1])
```

The target looks good. It only has values 0 and 1.

Time

In [16]:
```
df['Time'].hist()
```

Out[16]:
```
<Axes: >
```

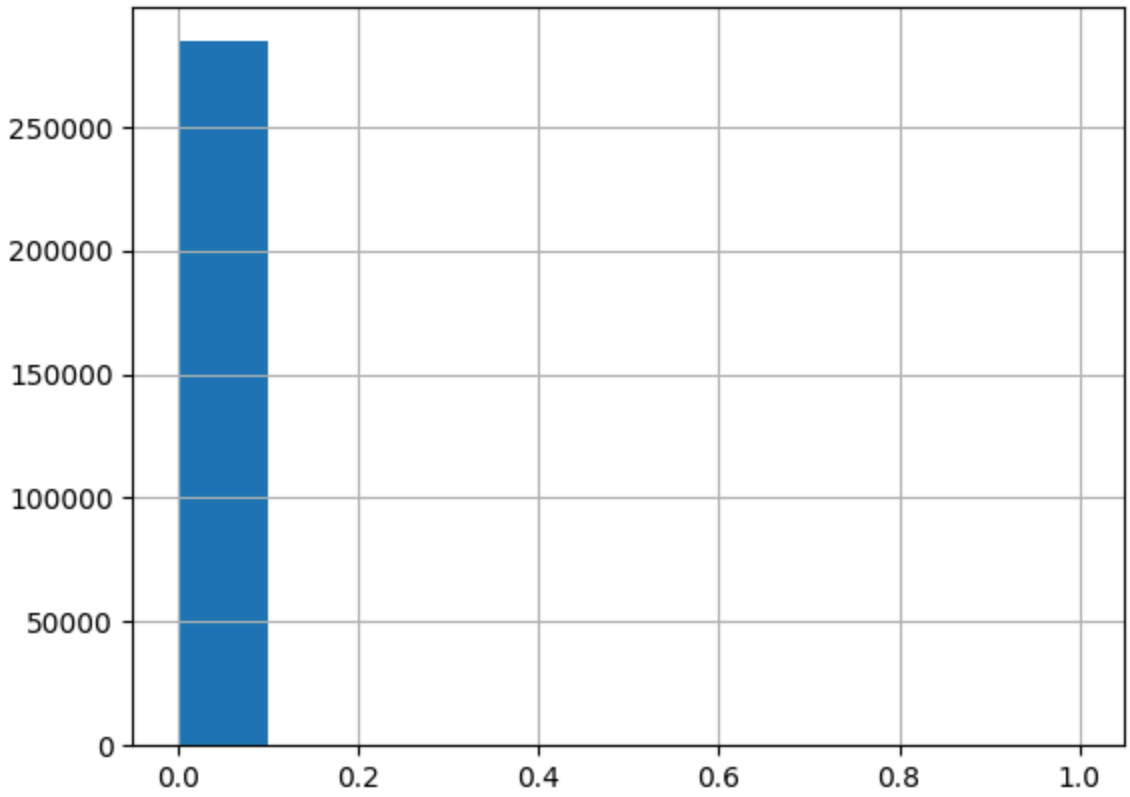


Interestingly, we can see that most of the instances are in the time range 30,000 - 85,000 and 120,000 - 175,000

Confirming Class Imbalance

In [17]:
```
df['Class'].hist()
```

Out[17]:
```
<Axes: >
```



In [18]:
```
len(df[df['Class'] == 1])
```

Out[18]: 492

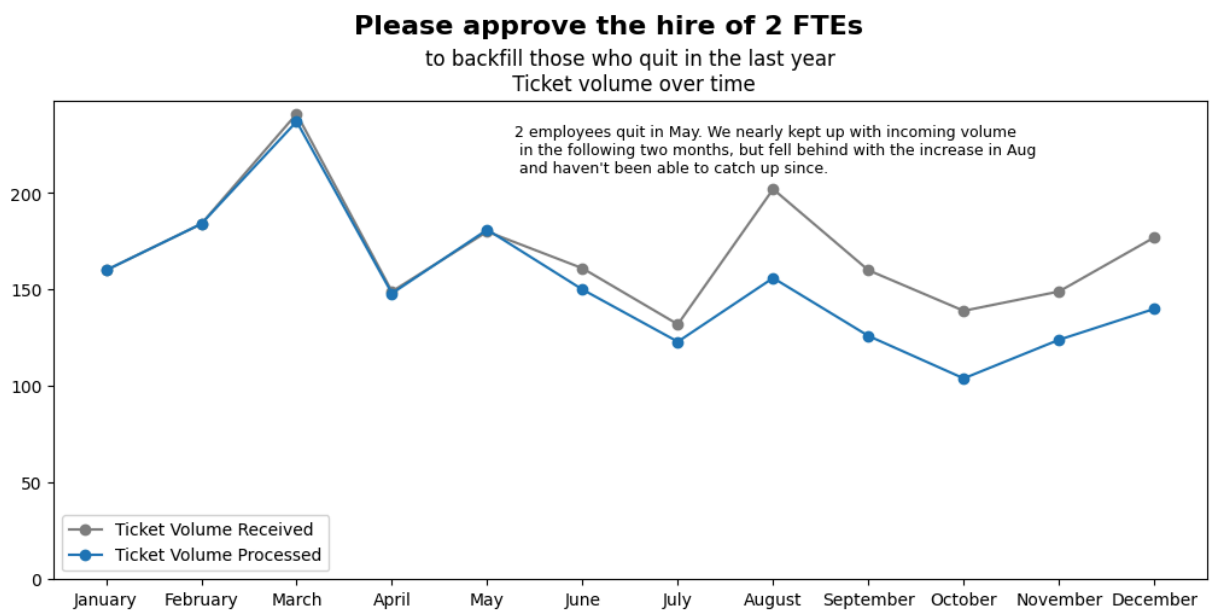
It is clear there is a huge class imbalance for the target variable (as is typical for fraud datasets). We will have to consider this throughout the project.

Conclusions

The data is mostly clean. There are duplicates in the dataset, which may need to be handled. Additionally, some of these duplicates have target value equal to 1. Since there is class imbalance for the target, we need to be careful with handling these duplicates. For the data, the transaction amounts do not have any strange or inconsistent data. Every value in this feature is numeric and positive. Additionally, all of the Time feature are positive, indicating the instances are recorded after the first record. We do see a few areas of the Time distribution which are dense, which could be interesting to explore further.

4. Storytelling With Data graph

Just like last week: choose any graph in the Introduction of Storytelling With Data (p. 1-17). Use matplotlib to reproduce it in a rough way. I don't expect you to spend an enormous amount of time on this; I understand that you likely will not have time to re-create every feature of the graph. However, if you're excited about learning to use matplotlib, this is a

good way to do that. You don't have to duplicate the exact values on the graph; just the same rough shape will be enough. If you don't feel comfortable using matplotlib yet, do the best you can and write down what you tried or what Google searches you did to find the answers.

In [61]:
```
# Figure 0.3
import matplotlib.pyplot as plt

ticket_data = {
    'Ticket Volume Received': (160, 184, 241, 149, 180, 161, 132, 202, 160, 139, 14
    'Ticket Volume Processed': (160, 184, 237, 148, 181, 150, 123, 156, 126, 104, 1
    'Month': ("January", "February", "March", "April", "May", "June", "July", "Augu
}
ticket_df = pd.DataFrame(ticket_data)
fig, ax = plt.subplots(layout='constrained', figsize=(10, 5))
plt.plot(ticket_df['Month'], ticket_df['Ticket Volume Received'], label="Ticket Vol
plt.plot(ticket_df['Month'], ticket_df['Ticket Volume Processed'], label="Ticket Vo
fig.suptitle('Please approve the hire of 2 FTEs', fontsize=16, fontweight='bold')
plt.title("to backfill those who quit in the last year\n Ticket volume over time")
ax.text(.4, .85, "2 employees quit in May. We nearly kept up with incoming volume \
plt.ylim(0)

plt.legend()
None
```